

Design Specifications Document

EagleNav – Campus Navigation Using Augmented Reality

Jian Mejia, Lcndr Aquino, George Barrone, Abigail Diaz,
Isidoro Flores, Santhosh Kumar, David Lopez, Reda Masri, Athena Ruiz

California State University, Los Angeles
Office for Students with Disabilities (OSD)

May 2026

Version	1.0
Date	May 2026
Sponsor	Office for Students with Disabilities (OSD)
Faculty Advisor	Dr. David Krum
Platform	Android 10+ / iOS 14+ — Flutter (Dart)
Status	Final – Academic Year 2025–2026

Contents

1 Project Overview

1.1 Purpose

EagleNav is an augmented reality campus navigation platform built for Android and iOS that uses custom high-precision mapping of the Cal State LA campus. The application is designed to support students with disabilities by offering accessible, independence-focused navigation tools including AR directional arrows, voice-guided directions, LiDAR-enhanced obstacle awareness, and seamless integration with campus events and building data. EagleNav aims to remove navigation barriers and create a more inclusive campus experience for all students, faculty, staff, and visitors at California State University, Los Angeles.

1.2 Scope

- Geographic coverage: Cal State LA main campus only – academic buildings, student services, housing perimeter paths, and all parking structures.
- Indoor coverage (v1): Major public corridors, elevators, ramps, and primary entrances/exits of high-traffic buildings.
- Off-campus routing is not supported.

1.3 Project Team

Name	Sub-team	Role
Jian Mejia	Computer Vision	Computer Vision Lead
Isidoro Flores	Computer Vision	Computer Vision
Santhosh Kumar	Computer Vision	Computer Vision
Abigail Diaz	Path Tracing	Path Tracing Lead
Reda Masri	Path Tracing	Path Tracing
George Barrone	Path Tracing	Path Tracing
Lcndr Aquino	Haptics	Haptics
Athena Ruiz	Haptics	Haptics
David Roberto Lopez	Data	Data

1.4 Key Stakeholders

- Project Sponsor: Office for Students with Disabilities (OSD), Cal State LA
- Faculty Advisor: Dr. David Krum
- Primary Users: CSULA students with disabilities (wheelchair, visually impaired, hearing impaired)
- Secondary Users: General student body, OSD staff, campus security, campus visitors

2 Design Goals and Principles

2.1 Core Design Goals

- **Accessibility First:** All features must be usable by students with visual, hearing, mobility, and cognitive disabilities.
- **Independence-Focused:** Navigation should empower users to navigate without requiring assistance from others.
- **Multimodal Feedback:** Every critical interaction must provide at minimum two of: visual, audio, and haptic feedback.
- **Graceful Degradation:** The app must function in degraded connectivity environments and fall back gracefully.
- **Minimal Cognitive Load:** UI complexity is strictly limited; interfaces must be intuitive without training.

2.2 Design Principles

- Deterministic routing: identical inputs always produce identical routes.
- Body-relative directions: all navigation cues reference the user's current facing direction, not cardinal directions.
- Redundant signals: every turn or alert is communicated via color + vibration + text.
- Independent voice channel: app speech is fully separate from system accessibility tools (VoiceOver / TalkBack).
- No gesture conflicts with OS accessibility layers.

3 System Architecture

3.1 Architectural Overview

EagleNav follows a three-tier layered architecture: Cloud Infrastructure, an on-device Controller Layer, and an on-device Service Layer. Each tier has a single responsibility, communicates only downward through well-defined interfaces, and is independently replaceable.

3.2 Cloud Infrastructure

- **Cloudflare R2:** Object storage hosting the campus OpenStreetMap extract (`renum_file.osm.pbf`). Valhalla loads this file into memory on startup.
- **Valhalla Routing Engine:** Open-source routing engine running on a Google Cloud Platform virtual machine; exposes a REST API consumed by the Flutter client.

3.3 Controller Layer (On-Device)

Controller	Responsibility
LocationController	Manages GPS stream, filters fixes by accuracy, exposes reactive current position.
RoutingController	Fetches routes from Valhalla, tracks deviation, triggers reroutes.
GuidanceController	Tracks step progression, advances steps at turn points, detects arrival.
NavigationVoiceController	Produces spoken instructions, manages TTS priority, exposes smoothed compass heading.
UiNavigationController	Owens four UI states: idle, destinationSelected, navigating, arrived.

3.4 Service Layer (On-Device)

Service	Responsibility
LocationService	Wraps geolocator plugin; exposes filtered position stream.
ValhallaService	Formats routing requests, posts over HTTPS, parses JSON response into typed step objects.
CompassService	Wraps flutter_compass; smooths magnetometer signal; exposes stable heading.
TTS Manager	Wraps flutter_tts with priority queue – urgent cues interrupt non-urgent ones.
HapticFeedback	Wraps device vibration API; produces patterned cues for turn events.
InstructionEnhancer	Converts Valhalla maneuver codes, distances, and angles into body-relative spoken phrases.
OrientationCoach	At route start, compares compass heading to first-segment bearing; produces alignment cues.
OffCourseDetector	Uses heading-aware thresholds (45m when aligned, 15m when opposed) for reroute decisions.
LandmarkService	On user request, computes nearby buildings and speaks their body-relative position.

3.5 Technology Stack

Layer / Component	Technology
Mobile Framework	Flutter 3.x / Dart 3.x (cross-platform iOS & Android)
Maps	flutter_map with OpenStreetMap tiles
Routing Engine	Valhalla (open-source) on Google Cloud Platform via REST
Map Data	OpenStreetMap extract (.osm.pbf) on Cloudflare R2
AR	ARCore (Android) / ARKit (iOS) via Flutter platform channels
Computer Vision (Android)	YOLO-based object detection via TensorFlow Lite
Computer Vision (iOS)	YOLO-based with CoreML; LiDAR hardware depth on supported devices
Local Storage	SharedPreferences (settings/bookmarks); JSON/GeoJSON assets
Security	TLS for all network calls; PII minimized; location processed on-device when possible

4 Data Flow Design

4.1 Route Planning Subsystem

#	Process	Description
P1.1	Building Selector	Resolves closest accessible entrance from buildings.json using current GPS.
P1.2	RoutingController	Assembles routing request and manages request lifecycle including reroutes.
P1.3	ValhallaService	Formats request as JSON, POSTs to Valhalla over HTTPS, parses response into typed route object.
P1.4	Valhalla Engine	Computes optimal pedestrian route with sidewalk preference and unpaved exclusion.

4.2 Guidance Subsystem

LocationController emits a filtered GPS position which the screen forwards to RoutingController and GuidanceController. RoutingController computes deviation using heading-aware thresholds (45m when aligned, 15m when opposed). GuidanceController checks step progression and advances the step index when the user passes a step endpoint. InstructionEnhancer converts maneuver codes and compass angles into body-relative spoken phrases, and HapticFeedback fires alongside each spoken cue.

4.3 Compass and Orientation Subsystem

The CompassService smooths magnetometer readings and exposes `currentHeading` as a reactive notifier. The OrientationCoach runs at navigation start and selects a coaching phrase based on the angle between the user's heading and the route's first segment:

Angle Range	Coaching Phrase
$ \text{angle} < 15$ degrees	Walk straight ahead
15 – 45 degrees	Slightly to your right / left
45 – 150 degrees	Turn to your right / left
$ \text{angle} > 150$ degrees	Turn around

4.4 Landmark Service Subsystem

The Landmark Service is user-initiated and operates independently of active navigation. It finds nearby buildings within a 50m radius, computes a body-relative bearing for each, converts metric distance to approximate step count (0.76m stride), and announces the result (e.g., “Library is to your front right, about 30 steps away.”).

4.5 Events Subsystem

The EventsScreen fetches HTML from the CSULA Events website, parses it into structured event data, and displays it to the user. Users can bookmark events stored locally via Shared-Preferences, and a notification scheduler fires local notifications for upcoming bookmarked events.

4.6 Computer Vision Subsystem

The camera captures real-time video input processed by YOLO-based detection and segmentation models. Detected objects are filtered by direction (left, ahead, right) and urgency, then delivered via TTS and haptics. Android uses a TensorFlow Lite depth estimator; iOS uses LiDAR hardware on supported devices with CoreML model equivalents otherwise.

5 Functional Requirements

5.1 General Application Behavior

- FR-0.1: Display the campus map as the default view on launch.
- FR-0.2: Provide access to Home, Favorites, Alerts, Profile, and Emergency Assistance from all views.
- FR-0.3: On launch, focus the search bar and announce via TTS: “Search for a destination.”

5.2 Destination Selection and Confirmation

- FR-1.1 / 1.2: Search for buildings by typing or speaking; results update dynamically.
- FR-2.1: Display generated route color-coded before navigation begins.
- FR-2.3: Display route summary: total distance, estimated travel time, and entrance name.

- FR-2.5: Show “Start Navigation” button after route summary is presented.

5.3 Core Navigation

- FR-3.1: Use a custom pedestrian path network for on-campus routing (Valhalla with campus OSM data).
- FR-3.2: At navigation start, provide turn-by-turn guidance and announce initial walking direction.
- FR-3.3: Alert the user of obstacles (stairs, blockages, construction).
- FR-3.4: Auto-recalculate route when the user deviates from the planned path.
- FR-3.5: Vibrate when off-route or approaching a turn.

5.4 Navigation Enhancements (v1.6)

- FR-14.1: Route to the closest accessible entrance from buildings.json.
- FR-14.2: Express all directions body-relative (e.g., “slightly to your right”), not cardinal.
- FR-14.3: Apply heading-aware deviation thresholds (45m when aligned, 15m when opposed).
- FR-14.4: Two-phase navigation start – Phase 1 guides to path network; Phase 2 begins turn-by-turn.
- FR-14.5: TTS priority queue – urgent cues (turn-now, arrival) interrupt non-urgent ones.
- FR-14.7: Express distances in approximate step counts, not meters.
- FR-14.8: Arrival detection at 8m from final step endpoint; announce door direction body-relative.
- FR-14.9: Valhalla pedestrian costing: prefer sidewalks/walkways, penalize alleys/tunnels, exclude unpaved.

5.5 Accessibility Requirements

5.5.1 Mobility Support

Large, high-contrast touch targets throughout. Voice commands for destination entry and navigation start. One-tap shortcuts to library, cafeteria, and OSD office. Prioritize ramps, elevators, and automatic doors in route generation.

5.5.2 Hearing Support

Distinct haptic patterns for left turn, right turn, straight, and wrong turn. Text mode for turn-by-turn instructions as readable text. Customizable vibration intensity and distance alerts.

5.5.3 Vision Support

Full compatibility with VoiceOver (iOS) and TalkBack (Android). Spoken descriptions of all menus, buttons, and navigation steps. Adjustable font sizes and zoom levels.

5.5.4 Cognitive Support

Simple, sequential navigation instructions. Landmark-based guidance (e.g., “Turn right at the fountain”). Optional confirmation after each instruction.

6 Non-Functional Requirements

Category	Requirement	Target
Performance	Route generation latency	95% of routes computed in less than 3 seconds
Performance	App cold start	Less than 2.5 seconds
Performance	UI frame budget	16ms target (60 fps)
Reliability	Core function uptime	99% uptime
Safety	Emergency button offline	Must function offline via SMS fallback
Security	Data encryption	End-to-end TLS; location data encrypted at rest and in transit
Security	PII minimization	No student records; location processed on-device when possible
Usability	Accessibility default	Accessibility features enabled on first run
Compatibility	OS support	Android 10+ and iOS 14+

7 User Interface Design

7.1 Design Principles

- Clean, minimal UI – reduces cognitive load and makes core functions immediately discoverable.
- Large touch targets – minimum 44x44pt (Apple HIG) for low-vision users.
- Optional high-contrast mode toggled from the accessibility profile.
- Customizable font sizes from the settings screen.
- Screen reader support as a first-class requirement, not an afterthought.

7.2 Screen Inventory

Screen	Key Elements
Home / Map	Search bar, campus map, current-location pin, bottom navigation bar
Route Preview	Color-coded route polyline, destination marker, route summary card, Start Navigation button
Navigation / Guidance	Turn banner, progress indicator, AR button, haptic/voice toggles, End/Cancel button
Favorites	Saved locations, quick-start navigation, edit/delete options
Events	Campus event list, bookmark toggle, local notification scheduler
Accessibility Profile	Profile selector, TTS speed/pitch/volume, haptic intensity, high-contrast toggle, font size slider
Emergency	One-touch campus security call, Safe Walk toggle, confirmation prompt

7.3 UI Navigation Flow

HomeMap → (Search or Favorite tap) → RoutePreview → Start → Guidance → (Deviation? → Recalculate) → Arrive → Done

8 Database and Data Design

8.1 Local Data Assets

File / Store	Purpose
buildings.json	Campus building names with accessible entrance coordinates.
buildings.geojson	Building footprints and entrance coordinates for map rendering.
bookmarks.json	User-saved routes and locations.
SharedPreferences	User accessibility preferences, app settings, and bookmarked events.
depth_estimator.tflite	Android TensorFlow Lite model for object distance estimation.
object_detector.tflite	Android TensorFlow Lite model for real-time YOLO object detection.
labels.txt	Classification label definitions for the object detection model.
renum_file.osm.pbf	Campus OpenStreetMap extract on Cloudflare R2.

8.2 Data Security Constraints

- All network calls over TLS; no plaintext transmission of location data.
- No student records stored or transmitted; PII minimized throughout.
- Sensitive data encrypted at rest on-device.
- Emergency location sharing transmits only to campus security / OSD, not third parties.

9 Implementation Status (v1.6)

FR	Feature Group	Status	Notes
FR-0	General Application Behavior	Implemented	Map default view, core navigation, TTS on launch.
FR-1	Destination Selection	Implemented	Search, dynamic results, map tap, clear.
FR-2	Destination Confirmation	Implemented	Route preview, summary, Start Navigation.
FR-3	Core Navigation	Implemented	Turn-by-turn, deviation detection, rerouting, haptics.
FR-4	Accessibility – Mobility	Partial	Voice commands, shortcuts, emergency profiles deferred.
FR-5	Accessibility – Hearing	Partial	Haptic turn patterns implemented. Custom settings deferred.
FR-6	Accessibility – Vision	Partial	Screen reader, font/zoom implemented. Voice input deferred.
FR-13	Speech Feedback (TTS)	Implemented	Full TTS pipeline implemented.
FR-14	Navigation Enhancements	Implemented	Closest-entrance, body-relative, heading-aware, two-phase start.
FR-15	Orientation & Awareness	Implemented	CompassService, Orientation-Coach, LandmarkService.
FR-9	Personalization & Scheduling	Future Work	All requirements deferred.
FR-12	Accessibility – Color Blind	Future Work	All color-blind themes deferred.

10 Testing and Validation Strategy

- **Unit Tests:** CompassService smoothing, InstructionEnhancer phrase generation, OffCourseDetector thresholds.
- **Integration Tests:** Route planning pipeline; Events parsing and notification scheduling.
- **Walk Tests:** End-to-end turn-by-turn guidance; reroute on deviation; arrival detection; LandmarkService with real buildings.
- **Manual Tests:** TTS priority interruption; VoiceOver / TalkBack compatibility; gesture non-conflict.
- **Performance Tests:** Route latency (less than 3s 95th percentile); cold start (less than 2.5s); UI frame time.

11 Future Work

11.1 Deferred Features

- Voice commands for destination entry and navigation initiation.
- Elevator/ramp preference in route costing.
- Emergency button with offline SMS fallback.
- Full accessibility user profiles – wheelchair, cane, temporary injury.
- Color-blind UI themes (protanopia, deuteranopia, tritanopia).
- Indoor room-to-room routing and wing/floor search.
- Personalization and scheduling – bookmarks, class schedules, SIS integration.

11.2 Potential Enhancements

- SIS integration to auto-import class schedules and provide next-class routing suggestions.
- Multi-language support (en-US baseline with right-to-left readiness).
- Wearable device integration for haptic-only navigation guidance.

12 Glossary

Term	Definition
AR	Augmented Reality – overlays digital navigation markers on the physical world via the device camera.
Body-relative direction	Direction expressed relative to the user’s current facing rather than cardinal direction.
CompassService	On-device service that smooths and validates raw magnetometer readings, exposing currentHeading.
GuidanceController	Real-time navigation processor tracking step progression and detecting arrival.
InstructionEnhancer	Converts Valhalla maneuver codes, distances, and angles into spoken body-relative phrases.
LandmarkService	Identifies nearby campus buildings and announces body-relative direction and step distance.
OffCourseDetector	Uses heading-aware thresholds to decide when to trigger a route recalculation.
OrientationCoach	One-shot service at navigation start aligning the user with the route’s first segment.
OSD	Office for Students with Disabilities at Cal State LA; primary project sponsor.
RoutingController	On-device controller managing route requests, active route state, and rerouting.
TTS	Text-to-Speech – converts navigation instructions to spoken audio.
Valhalla	Open-source routing engine hosted on GCP, computing pedestrian routes from OSM data.

13 References

1. EagleNav Software Requirements Specification (SRS) v1.6 – OSD, Cal State LA, May 2026
2. EagleNav Software Design Document (SDD) v4.0 – OSD, Cal State LA, May 2026
3. IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications
4. CSULA Campus Map. <https://www.calstatela.edu/map>
5. Valhalla Routing Engine. <https://github.com/valhalla/valhalla>
6. Flutter / Dart Documentation. <https://docs.flutter.dev>
7. Apple Human Interface Guidelines – Accessibility
8. Android Accessibility Developer Guide – TalkBack & Touch Targets